

ShipBud

Design Document

Version 2.0 | March 22, 2026

Version	Date	Changes	Author
1.0	Mar 22, 2026	Initial design document	Solo Dev
2.0	Mar 22, 2026	Prisma Postgres replaces Neon, Stripe added, constraints, landing page plan	Solo Dev

1. System Overview

ShipBud is a Next.js 14 App Router web application that wraps the entire SDLC for solo student developers. It is backed by Prisma Postgres (hosted database + ORM in one), automated by n8n background workflows, secured by Auth0 (Token Vault + M2M), uses the Vercel AI SDK with Featherless AI for all AI features, and Stripe for subscription billing.

The system is event-driven. GitHub webhooks and n8n polling schedules drive most state changes without requiring user intervention. The frontend communicates exclusively with Next.js API routes, which interact with Prisma Postgres, Auth0, Featherless AI, and Stripe.

2. Final Technology Stack

Layer	Technology	Notes
Framework	Next.js 14 (App Router)	API routes + frontend in one project
UI	shadcn/ui + Tailwind CSS	Component library on top of Radix UI
Auth + Token Vault	Auth0	Identity, OAuth tokens, M2M, CIBA
Database + ORM	Prisma Postgres + Prisma ORM	Hosted Postgres + ORM from same provider. <code>npx prisma init --db</code>
AI Provider	Featherless AI	Base URL: <code>api.featherless.ai/v1</code> — Premium tier

Layer	Technology	Notes
AI Model	Qwen/Qwen2.5-72B-Instruct	Strong structured JSON output
AI SDK	Vercel AI SDK v4 + @ai-sdk/openai-compatible	streamText, generateObject, useChat, useCompletion
Payments	Stripe	Subscription billing, Checkout, webhooks, Customer Portal
Automation	n8n	Drift detection, commit processing, Todoist sync
Deployment	Vercel	Edge-compatible, Auth0 + Prisma native

3. System Architecture

Layer Overview

- Client: Next.js 14 App Router + shadcn/ui — renders pages, calls API routes
- API Routes: /api/* — all business logic, data access, AI calls, webhook handling
- Services: Prisma Postgres (data), Auth0 (identity + tokens), Featherless AI (AI), Stripe (billing)
- Background: n8n — drift poller, Todoist sync, commit processor

Request Flow — User Action

1. Browser calls Next.js API route with Auth0 session JWT
2. API route validates JWT via Auth0, extracts ownerId
3. Subscription plan checked against Prisma DB for gated features
4. Prisma query scoped to ownerId — data isolation at ORM layer
5. Response returned (streamed via Vercel AI SDK where applicable)

Request Flow — n8n Agent

6. n8n cron fires → GET /api/agents/projects with Auth0 M2M service key
7. API validates M2M token, returns active projects in DEV+ phase
8. n8n loops projects → GET /api/agents/activity/:id
9. If drift detected → POST /api/agents/drift/:id → email sent via n8n

Request Flow — GitHub Webhook

10. GitHub POSTs to /api/webhooks/github on every commit
11. API validates HMAC signature using repo webhook secret
12. Commit extracted → auto-draft journal entry created (status: DRAFT)
13. ActivityLog updated → inactivity clock reset

Request Flow — Stripe Webhook

14. Stripe POSTs to /api/webhooks/stripe on billing events

15. API reads raw body, validates Stripe signature (`stripe.webhooks.constructEvent`)
16. Idempotency check: `stripeEventId` already in `StripeEvent` table? Skip.
17. Process event: update Subscription record in Prisma accordingly

4. Architectural Constraints & Resolutions

These constraints were identified during design and must be addressed during implementation. Each has a defined resolution that is reflected in the architecture.

C-01 — Prisma on Vercel Runtime

[CONSTRAINT] Prisma's default client requires Node.js runtime. Edge runtime will fail.

[RESOLUTION] All API routes that use Prisma must explicitly set: `export const runtime = 'nodejs'`. Never use Edge runtime for data routes.

C-02 — n8n Needs a Live Public URL

[CONSTRAINT] n8n calls `/api/agents/*` on a schedule. It cannot reach localhost during development.

[RESOLUTION] Deploy to Vercel at the end of Day 3 after foundation is built. Use Vercel preview URL in n8n during development. Deployment is a Day 3 task, not Day 7.

C-03 — GitHub Webhooks Need a Live URL

[CONSTRAINT] GitHub cannot POST commit events to localhost.

[RESOLUTION] Same fix as C-02 — deploy on Day 3. Alternatively use ngrok locally for Days 3-4 only, then switch to Vercel preview URL.

C-04 — Auth0 Token Vault OAuth App Approval

[CONSTRAINT] Twitter and Todoist OAuth apps must be registered and may require approval time.

[RESOLUTION] Register Twitter and Todoist OAuth apps on Day 3 immediately. If Twitter approval is delayed, content export degrades gracefully to copy-to-clipboard. Still demos well.

C-05 — Stripe Webhook Signature Verification

[CONSTRAINT] Without `constructEvent()` verification, anyone can POST fake events and upgrade themselves to Pro for free.

[RESOLUTION] Always verify raw request body against Stripe webhook secret. In Next.js App Router: use `req.text()` not `req.json()` in the webhook handler before passing to `stripe.webhooks.constructEvent()`.

C-06 — Stripe Trial Expiry State Sync

[CONSTRAINT] When a trial expires, Stripe fires `customer.subscription.updated`. If unhandled, users stay on Pro forever.

[RESOLUTION] Handle exactly three Stripe webhook events: `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`. These three cover the full subscription lifecycle.

C-07 — Feature Gating Must Be Server-Side

[CONSTRAINT] UI-only gating can be bypassed by calling API routes directly.

[RESOLUTION] All gated features (drift detection, content generation, unlimited projects) are checked at the API route level against the user's plan field in Prisma — not just in the frontend.

C-08 — Stripe Webhook Idempotency

[CONSTRAINT] Stripe can deliver the same webhook event more than once. Duplicate processing will corrupt subscription state.

[RESOLUTION] Store stripeEventId on every processed event in a StripeEvent table. Before processing, check if the event ID exists. If yes, return HTTP 200 and skip processing.

5. Prisma Schema

Setup — Prisma Postgres

```
# Day 3 setup command — creates database + initialises Prisma in one step
npx prisma init --db
```

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

Enums

```
enum ProjectStatus { ACTIVE COMPLETED ARCHIVED }
enum PhaseType     { PLANNING DESIGN DEV TESTING DEPLOY }
enum PhaseStatus   { LOCKED ACTIVE COMPLETED }
enum ItemStatus    { TODO IN_PROGRESS DONE }
enum ItemSource    { AI USER }
enum EntryType     { OBSTACLE WIN DECISION }
enum EntrySource   { MANUAL AUTO_DRAFT }
enum EntryStatus   { DRAFT CONFIRMED }
enum ActivityType  { COMMIT JOURNAL_ENTRY }
enum DriftStatus   { WATCHING DRIFTED SUPPRESSED }
enum DeliveryStatus { SENT FAILED }
enum ContentSource { PHASE FULL_PROJECT OBSTACLE }
enum ContentFormat { TWITTER_POST README BUILD_STORY OBSTACLE_POST }
enum DraftStatus   { DRAFT EDITED EXPORTED }
enum PlanType      { FREE PRO }
```

Core Models

Project

```
model Project {
  id          String      @id @default(uuid())
  ownerId     String
  name        String
  description  String?
  targetDate  DateTime?
  status      ProjectStatus @default(ACTIVE)
  createdAt   DateTime     @default(now())

  repo        Repo?
  phases      Phase[]
  journalEntries JournalEntry[]
}
```

```

    activityLog      ActivityLog?
    driftMonitor     DriftMonitor?
    contentRequests ContentRequest[]
}

```

Repo

```

model Repo {
  id          String    @id @default(uuid())
  projectId   String    @unique
  githubRepoUrl String
  webhookId   String
  connectedAt DateTime @default(now())
  project     Project    @relation(fields: [projectId], references: [id], onDelete: Cascade)
}

```

Phase

```

model Phase {
  id          String    @id @default(uuid())
  projectId   String
  type        PhaseType
  status      PhaseStatus @default(LOCKED)
  position    Int
  project     Project    @relation(fields: [projectId], references: [id], onDelete: Cascade)
  milestones  Milestone[]
}

```

Milestone

```

model Milestone {
  id          String    @id @default(uuid())
  phaseId     String
  title       String
  status      ItemStatus @default(TODO)
  source      ItemSource @default(AI)
  position    Int
  phase       Phase      @relation(fields: [phaseId], references: [id], onDelete: Cascade)
  tasks       Task[]
  journalEntries JournalEntry[]
}

```

Task

```

model Task {
  id          String    @id @default(uuid())
  milestoneId String
  title       String
  status      ItemStatus @default(TODO)
  source      ItemSource @default(AI)
}

```

```

    position      Int
    milestone Milestone @relation(fields: [milestoneId], references: [id], onDelete:
Cascade)
}

```

JournalEntry

```

model JournalEntry {
  id          String      @id @default(uuid())
  projectId   String
  phaseId     String?
  milestoneId String?
  type        EntryType
  source       EntrySource @default(MANUAL)
  status       EntryStatus @default(CONFIRMED)
  content      Json        // shape varies by EntryType (see below)
  createdAt    DateTime    @default(now())
  project      Project      @relation(fields: [projectId], references: [id], onDelete:
Cascade)
  milestone Milestone? @relation(fields: [milestoneId], references: [id])
}

```

The content Json field shape by EntryType:

- OBSTACLE: { problem: string, root_cause: string, solution: string }
- WIN: { description: string }
- DECISION: { description: string, rationale: string }

ActivityLog + ActivityEvent

```

model ActivityLog {
  id          String      @id @default(uuid())
  projectId   String      @unique
  lastActivityAt DateTime
  lastActivityType ActivityType
  project      Project      @relation(fields: [projectId], references: [id], onDelete:
Cascade)
  events      ActivityEvent[]
}

```

```

model ActivityEvent {
  id          String      @id @default(uuid())
  activityLogId String
  type        ActivityType
  occurredAt   DateTime
  metadata     String?    // commit SHA or journalEntryId
  activityLog ActivityLog @relation(fields: [activityLogId], references: [id],
onDelete: Cascade)
}

```

DriftMonitor + DriftNotification

```

model DriftMonitor {
  id          String      @id @default(uuid())

```



```

    projectId      String      @unique
    thresholdHours Int          @default(24)
    status          DriftStatus @default(WATCHING)
    lastCheckedAt   DateTime?
    project          Project      @relation(fields: [projectId], references:
[id], onDelete: Cascade)
    notifications   DriftNotification[]
}

model DriftNotification {
  id              String          @id @default(uuid())
  driftMonitorId String
  sentAt          DateTime
  lastActivityAt  DateTime
  currentPhase    PhaseType
  deliveryStatus  DeliveryStatus @default(SENT)
  driftMonitor    DriftMonitor @relation(fields: [driftMonitorId], references: [id],
onDelete: Cascade)
}

```

ContentRequest + ContentDraft

```

model ContentRequest {
  id              String          @id @default(uuid())
  projectId       String
  source          ContentSource
  sourceId        String
  formats         ContentFormat[]
  requestedAt     DateTime        @default(now())
  project         Project          @relation(fields: [projectId], references: [id], onDelete:
Cascade)
  drafts          ContentDraft[]
}

model ContentDraft {
  id              String          @id @default(uuid())
  contentRequestId String
  format          ContentFormat
  body            String
  editedBody      String?
  status          DraftStatus    @default(DRAFT)
  contentRequest  ContentRequest @relation(fields: [contentRequestId], references:
[id], onDelete: Cascade)
}

```

Stripe Models

Subscription

```

model Subscription {
  id              String          @id @default(uuid())
  ownerId         String          @unique
  stripeCustomerId String        @unique
}

```

```
stripeSubId      String    @unique
plan             PlanType  @default(FREE)
status           String    // active | trialing | past_due | canceled
trialEndsAt      DateTime?
currentPeriodEnd DateTime?
createdAt        DateTime  @default(now())
updatedAt        DateTime  @updatedAt
}
```

StripeEvent (idempotency table)

```
model StripeEvent {
  id      String  @id // Stripe event ID - used for idempotency check
  createdAt DateTime @default(now())
}
```

6. Stripe Integration

Pricing Model

Plan	Price	Projects	Drift Detection	Content Generation	Todoist Sync
Free Trial	Free — 14 days	1 project	Enabled	Enabled	Enabled
Pro	\$9 / month	Unlimited	Enabled	Enabled	Enabled
Free (expired)	\$0	1 project	Disabled	Disabled	Disabled

Trial starts automatically on signup. No credit card required. After 14 days, gated features are disabled until the user upgrades to Pro.

Stripe Webhook Event Map

Stripe Event	Action in ShipBud	Priority
checkout.session.completed	Create Subscription record. Set plan: PRO, status: trialing, trialEndsAt from Stripe session.	Required
customer.subscription.updated	Sync status and currentPeriodEnd to Subscription record. Handle trial expiry (status -> past_due).	Required
customer.subscription.deleted	Set plan: FREE, status: canceled on Subscription record.	Required

Webhook Handler Pattern

```
// /api/webhooks/stripe/route.ts
export const runtime = 'nodejs'; // required — must read raw body

const rawBody = await req.text(); // NOT req.json()
const sig = req.headers.get('stripe-signature')!;
const event = stripe.webhooks.constructEvent(rawBody, sig,
process.env.STRIPE_WEBHOOK_SECRET!);

// Idempotency check
const existing = await prisma.stripeEvent.findUnique({ where: { id: event.id } });
if (existing) return new Response('OK', { status: 200 });

// Process event
await prisma.stripeEvent.create({ data: { id: event.id } });
```

```
switch (event.type) { ... }
```

Feature Gating Pattern

```
// lib/gate.ts - call at the start of any gated API route
export async function requirePro(ownerId: string) {
  const sub = await prisma.subscription.findUnique({ where: { ownerId } });
  const isActive = sub?.status === 'active' || sub?.status === 'trialing';
  if (!isActive) throw new Error('UPGRADE_REQUIRED');
}
```

7. API Routes

Projects

Method	Route	Description
POST	/api/projects	Create project + init phases + activity log + drift monitor
GET	/api/projects	List all projects for authenticated user
GET	/api/projects/:id	Get project + phases + milestones + tasks
PATCH	/api/projects/:id	Update name, description, target date, status
POST	/api/projects/:id/repo	Connect GitHub repo + register webhook via Auth0 Token Vault
POST	/api/projects/:id/plan	Trigger AI SDLC plan generation (streams response)

Milestones & Tasks

Method	Route	Description
POST	/api/milestones	Create milestone under a phase
PATCH	/api/milestones/:id	Update title, status, position — enforces INV-04
DELETE	/api/milestones/:id	Delete milestone + cascade tasks
POST	/api/tasks	Create task under a milestone
PATCH	/api/tasks/:id	Update title, status, position
DELETE	/api/tasks/:id	Delete task

Journal

Method	Route	Description
POST	/api/journal	Create journal entry — enforces INV-01 and INV-05
GET	/api/journal?project_id=	List all entries for a project
PATCH	/api/journal/:id	Edit entry or confirm auto-draft
DELETE	/api/journal/:id	Delete entry

Content (Pro-gated)

Method	Route	Description
POST	/api/content/request	Trigger content generation — enforces INV-03 + requirePro()

Method	Route	Description
GET	/api/content/:request_id	Get all drafts for a content request
PATCH	/api/content/drafts/:id	Edit a draft body
POST	/api/content/drafts/:id/export	Mark draft as exported

Stripe

Method	Route	Description
POST	/api/stripe/checkout	Create Stripe Checkout session for Pro plan
GET	/api/stripe/portal	Create Stripe Customer Portal session for billing management
POST	/api/webhooks/stripe	Receive Stripe events — raw body + signature verification

Webhooks & Agent Routes

Method	Route	Auth	Description
POST	/api/webhooks/github	GitHub HMAC signature	Receive commit events → auto-draft + activity reset
GET	/api/agents/projects	Auth0 M2M service key	n8n fetches active projects in DEV+ phase
GET	/api/agents/activity/:id	Auth0 M2M service key	n8n checks last activity timestamp + threshold
POST	/api/agents/drift/:id	Auth0 M2M service key	n8n reports drift → create notification record

8. AI Integration

Provider Setup

```
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';
import { generateObject, streamText } from 'ai';

const featherless = createOpenAICompatible({
  name: 'featherless',
  apiKey: process.env.FEATHERLESS_API_KEY,
  baseUrl: 'https://api.featherless.ai/v1',
});

const model = featherless('Qwen/Qwen2.5-72B-Instruct');
```

Feature	SDK Method	Output Strategy	Gated
SDLC Plan Generation	generateObject	Zod schema → typed Phase/Milestone/Task tree	No
Commit Auto-Draft	generateObject	Zod schema → typed JournalEntry draft	No
Obstacle Structuring	generateObject	Zod schema → { problem, root_cause, solution }	No
Content Generation	streamText	Streamed to client via useCompletion hook	Pro
SDLC Chat Guidance	streamText	Streamed to client via useChat hook	No

9. n8n Workflow Design

Workflow 1 — Drift Poller (every 2 hours, Pro users only)

18. Cron trigger fires every 2 hours
19. HTTP GET /api/agents/projects — fetch active projects in DEV/TESTING/DEPLOY phase with Pro plan
20. Loop: HTTP GET /api/agents/activity/:id — get lastActivityAt per project
21. IF now() - lastActivityAt > thresholdHours: POST /api/agents/drift/:id
22. Auth0 Token Vault: fetch user email for notification delivery
23. Send email: last activity date, current phase, original project goal, link back to project

Workflow 2 — Todoist Sync (manual trigger, Pro users only)

24. User clicks Sync to Todoist button on project plan page
25. HTTP GET /api/projects/:id — fetch milestones and tasks
26. Auth0 Token Vault: fetch user Todoist OAuth token

27. Todoist API: create project sections (milestones) and tasks under each section

10. Auth0 Integration

Concern	Auth0 Feature	Notes
User login	Universal Login	Email + GitHub social sign-in
GitHub OAuth token	Token Vault — connection: github	Webhook registration + commit reads
Twitter OAuth token	Token Vault — connection: twitter	Content export — degrades to clipboard if unavailable
Todoist OAuth token	Token Vault — connection: todoist	Used by n8n Workflow 2
n8n agent auth	Machine-to-Machine client	Service key in Authorization header
Drift approval	CIBA	Out-of-band push approval for high-stakes actions

11. Landing Page Plan

Route: / — built in Next.js App Router, same repo, no separate deployment. Built on Day 6 using shadcn/ui components.

Copy

- **Headline:** Stop abandoning projects. Start shipping them.
- **Subheadline:** ShipBud wraps your entire build process — from idea to shipped — and keeps you on track when momentum fades.
- **Primary CTA:** Start building free → Auth0 signup
- **Trial CTA:** 14-day free trial. No credit card required.

Section Breakdown

#	Section	Content	shadcn Components
1	Hero	Headline, subheadline, primary CTA button, subtle background gradient	Button, Badge
2	Problem	3 pain points with icons: I start but never finish / I lose momentum / Nobody sees what I build	Card
3	How It Works	3-step numbered flow: Describe your idea → Build with AI guidance → Ship + tell the story	Custom numbered steps
4	Feature Highlights	4 feature cards: SDLC Planner, Build Journal, Drift Detection, Content Generation	Card, Badge
5	Demo Video	Embedded video player (YouTube or direct mp4). Placeholder until Day 28 recording.	Aspect-ratio container
6	Pricing	Two columns: Free Trial vs Pro \$9/mo. Feature comparison. Upgrade CTA.	Card, Button, Separator
7	Footer	Built with: Next.js, Auth0, n8n, Prisma, Featherless AI. GitHub link.	Separator

12. Frontend Page Structure

Route	Page	Key Components	Plan Gate
/	Landing	Hero, Problem, How It Works, Features, Demo, Pricing, Footer	Public

Route	Page	Key Components	Plan Gate
/dashboard	All Projects	Project cards, status badge, last activity, create new CTA	Any
/projects/new	Create Project	Name/description form, AI plan generation stream	Any (1 project limit on Free expired)
/projects/:id	Project Overview	Summary, repo connection status, quick stats	Any
/projects/:id/plan	SDLC Plan	Phase timeline, milestone list, task checklist, progress map	Any
/projects/:id/journal	Build Journal	Entry list, create form, auto-draft confirm card	Any
/projects/:id/content	Content Hub	Content request trigger, draft editor, export buttons	Pro
/settings	Settings	GitHub, Twitter, Todoist connection status + revoke, billing portal link	Any
/pricing	Pricing	Standalone pricing page, Stripe Checkout trigger	Public

13. Domain Invariants

ID	Rule	Enforced In
INV-01	JournalEntry milestone must belong to the entry's project	POST /api/journal
INV-02	DriftMonitor is suppressed when current phase is PLANNING or DESIGN	n8n Workflow 1 filter
INV-03	ContentRequest of source OBSTACLE must reference an OBSTACLE journal entry	POST /api/content/request
INV-04	Phase cannot be marked COMPLETED if any milestone is not DONE	PATCH /api/milestones/:id
INV-05	AUTO_DRAFT journal entry cannot be CONFIRMED without explicit user action	PATCH /api/journal/:id
INV-06	Drift detection and content generation only run for Pro or trialing users	requirePro() middleware
INV-07	Stripe webhook events must be idempotent — duplicate event IDs are skipped	POST /api/webhooks/stripe

14. Revised Build Timeline

Day	Date	Focus	Key Tasks
Day 2	Mar 22	Planning & Design (today)	SDLC docs, DDD, Design Document v2.0, RTM
Day 3	Mar 23	Foundation + Deploy Early	Auth0, Prisma Postgres init, GitHub OAuth, webhook, register Twitter/Todoist OAuth apps, deploy to Vercel preview
Day 4	Mar 24	Core Build Loop	SDLC planner (AI generation), project dashboard, progress map
Day 5	Mar 25	Journal + Drift	Build journal (manual + auto-draft), n8n drift detection, email notification
Day 6	Mar 26	Content + Stripe + Landing + Polish	Content generation, Stripe vibe code, landing page, bug fixes, seed demo data
Day 7	Mar 27	Test + Deploy Production	Full test pass, Vercel production deploy, n8n production workflow, smoke test all flows
Day 8	Mar 28	Submission	Record 2-3 min demo video, Devpost write-up, submit before deadline

15. Environment Variables

Variable	Used By	Description
DATABASE_URL	Prisma	Prisma Postgres connection string (from npx prisma init --db)
AUTH0_SECRET	Auth0 SDK	Session encryption secret
AUTH0_BASE_URL	Auth0 SDK	App base URL — Vercel deployment URL
AUTH0_ISSUER_BASE_URL	Auth0 SDK	Auth0 tenant URL
AUTH0_CLIENT_ID	Auth0 SDK	App client ID
AUTH0_CLIENT_SECRET	Auth0 SDK	App client secret
AUTH0_M2M_CLIENT_ID	n8n	Machine-to-machine client ID for agent auth
AUTH0_M2M_CLIENT_SECRET	n8n	Machine-to-machine client secret
FEATHERLESS_API_KEY	Vercel AI SDK	Featherless AI premium API key
GITHUB_WEBHOOK_SECRET	Webhook handler	HMAC secret for validating GitHub payloads
N8N_SERVICE_API_KEY	Agent routes	Secret key n8n passes to /api/agents/* routes
STRIPE_SECRET_KEY	Stripe SDK	Stripe secret key (sk_live_* or sk_test_*)
STRIPE_WEBHOOK_SECRET	Webhook handler	Stripe webhook signing secret (whsec_*)
STRIPE_PRO_PRICE_ID	Checkout	Stripe price ID for the Pro \$9/mo plan
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Frontend	Stripe publishable key for client-side Stripe.js